

Danh sách liên kết đôi



LINKED LIST

• TS Phạm Hữu Tùng

DOUBLY LINKED LIST



Real Python

Nguồn hình ảnh: Linked Lists in Python: An Introduction

1. Danh sách liên kết đôi là gì?

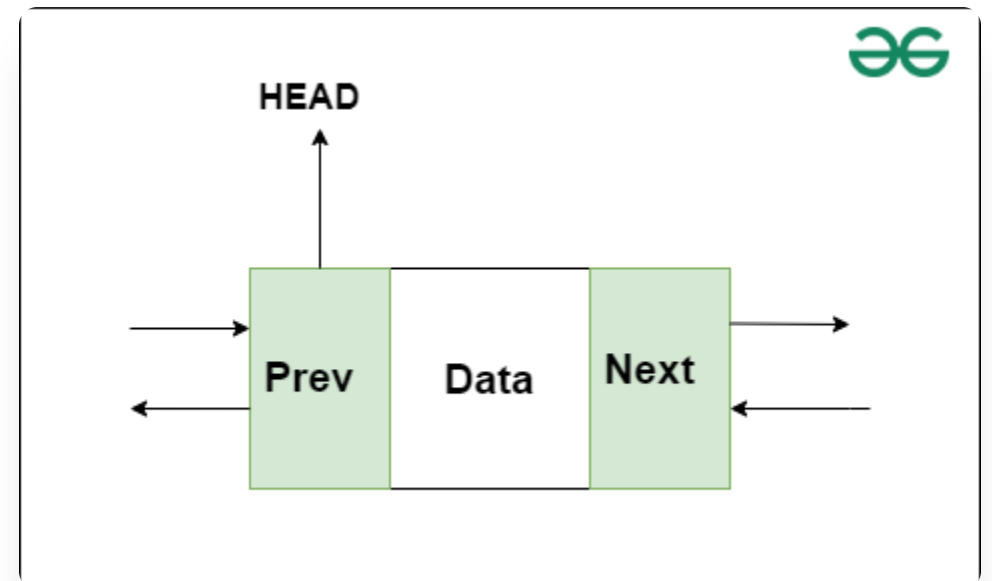
📄 Định nghĩa cơ bản

- Là một cấu trúc dữ liệu tuyến tính, bao gồm một chuỗi các "nút" (nodes) được liên kết tuần tự với nhau.
- Điểm đặc trưng là mỗi nút chứa dữ liệu và **hai con trỏ**, cho phép duyệt danh sách theo cả hai chiều: tiến và lùi.
- Cung cấp sự linh hoạt hơn so với danh sách liên kết đơn trong việc chèn và xóa phần tử.

🔧 Cấu trúc của một Nút (Node)

Mỗi nút trong danh sách liên kết đôi gồm ba thành phần chính:

- prev** Con trỏ trỏ đến **trước** . Nút đầu tiên (head) có `prev` nút đứng **nó** là `NULL`.
- data** Phần tử dữ liệu thực tế mà nút lưu trữ (ví dụ: số, chuỗi, đối tượng).
- next** Con trỏ trỏ đến nút **sau** . Nút cuối cùng (tail) có `next` đứng **nó** là `NULL`.



Cấu trúc một nút và liên kết trong danh sách liên kết đôi.

Khác biệt so với Danh sách liên kết đơn và Mảng

↔ So với Danh sách liên kết đơn

- ↔ **Duyệt:** Hai chiều (tiến/lùi) so với chỉ một chiều.
- 📦 **Bộ nhớ:** Tốn nhiều hơn do có thêm con trỏ `prev`.
- 🔄 **Thao tác:** Xóa nút hiệu quả hơn (không cần duyệt để tìm nút trước đó).

☰ So với Mảng

- 📏 **Kích thước:** Động, có thể thay đổi linh hoạt so với kích thước cố định.
- 📍 **Lưu trữ:** Các nút phân tán trong bộ nhớ so với khối bộ nhớ liên tục.
- 🕒 **Truy cập:** Tuần tự ($O(N)$) so với truy cập ngẫu nhiên ($O(1)$).

2. Ưu và nhược điểm

✓ Ưu điểm

- » **Duyệt hai chiều:** Dễ dàng di chuyển tiến và lùi trong danh sách, tăng cường khả năng điều hướng và tìm kiếm linh hoạt.
- ≡ **Dễ dàng xóa nút:** Loại bỏ nút đơn giản hơn vì chỉ cần cập nhật con trỏ của các nút lân cận, không cần tham chiếu nút trước đó.
- ⚙ **Quản lý bộ nhớ động:** Kích thước danh sách có thể thay đổi linh hoạt trong quá trình chạy, không bị giới hạn bởi kích thước ban đầu.



Nhược điểm

- ≡ **Tốn thêm không gian:** Mỗi nút yêu cầu thêm một con trỏ `prev`, làm tăng yêu cầu bộ nhớ so với danh sách liên kết đơn.
- ⚡ **Phức tạp hơn trong triển khai:** Các thao tác thêm, xóa đòi hỏi cập nhật cả hai con trỏ (`prev` và `next`), dễ phát sinh lỗi.

So sánh hiệu quả

So với Danh sách liên kết đơn: DLL cho phép duyệt hai chiều và xóa nút $O(1)$ (nếu có con trỏ đến nút), nhưng tốn nhiều bộ nhớ hơn.

So với Mảng: DLL cung cấp kích thước động và chèn/xóa $O(1)$ hiệu quả, trong khi mảng có truy cập ngẫu nhiên nhanh hơn $O(1)$ nhưng chèn/xóa chậm $O(n)$.

💡 Trường hợp sử dụng phù hợp

Hệ thống Undo/Redo: Trong các trình soạn thảo, cho phép quay lại hoặc tiến tới các trạng thái.

Lịch sử duyệt web: Quản lý các trang đã truy cập để di chuyển tiến/lùi.

Quản lý bộ nhớ đệm (Cache): Triển khai các thuật toán như LRU (Least Recently Used).

URL	Title	Visit Time	Visit Count	Web browser	User Profile
http://espn.go.com/	ESPN: The Worldwide Lea...	19/08/12 13:45:34	1	Safari	Adminis...
http://www.apple.com/st...	Apple - Start	19/08/12 13:45:12	2	Safari	Adminis...
http://www.google.com		19/08/12 13:44:01	43	Internet Explorer	Adminis...
http://www.google.com		19/08/12 13:44:01	42	Internet Explorer	Adminis...
https://accounts.google...	Gmail: Email from Google	19/08/12 13:42:52	1	Chrome	Adminis...
https://mail.google.com/...	Gmail: Email from Google	19/08/12 13:42:52	1	Chrome	Adminis...
https://www.gmail.com/	Gmail: Email from Google	19/08/12 13:42:52	1	Chrome	Adminis...
http://www.gmail.com/	Gmail: Email from Google	19/08/12 13:42:52	1	Chrome	Adminis...
http://www.facebook.com/	Welcome to Facebook - L...	19/08/12 13:42:39	1	Chrome	Adminis...
http://www.windowsmedi...	Windows Media Guide H...	19/08/12 13:42:23	4	Firefox	Adminis...
http://www.windowsmedi...		19/08/12 13:42:22	4	Firefox	Adminis...
http://www.windowsmedi...		19/08/12 13:42:22	5	Firefox	Adminis...

Ví dụ: Lịch sử duyệt web

3. Các thao tác cơ bản

NEW

Khởi tạo Danh sách (List Initialization)

- Để khởi tạo một danh sách liên kết đôi trống, con trỏ `head` và `tail` được đặt là `NULL`.
- Điều này biểu thị rằng danh sách hiện không chứa bất kỳ nút nào và sẵn sàng để thêm phần tử.
- Khi nút đầu tiên được thêm vào, cả `head` và `tail` đều trỏ tới nút đó.

+

Thêm nút (Insertion)

◀

Thêm vào đầu:

- Tạo nút mới, đặt `next` của nó trỏ tới `head` hiện tại.
- Nếu danh sách không rỗng, đặt `prev` của `head` hiện tại trỏ tới nút mới.
- Cập nhật `head` thành nút mới.

>|

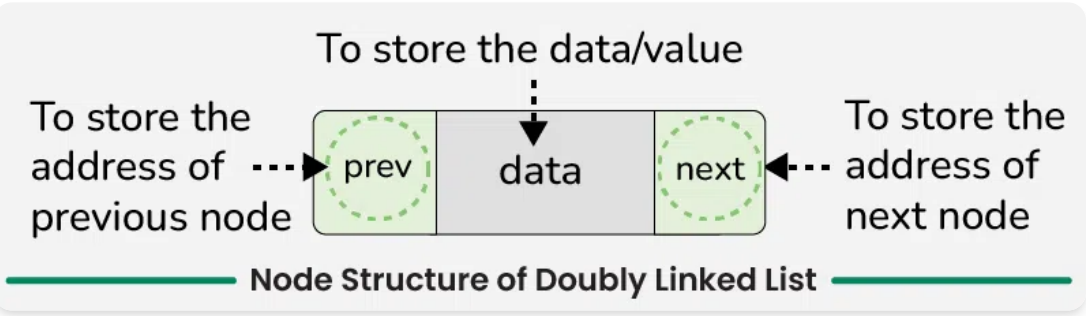
Thêm vào cuối:

- Tạo nút mới, đặt `prev` của nó trỏ tới `tail` hiện tại.
- Nếu danh sách không rỗng, đặt `next` của `tail` hiện tại trỏ tới nút mới.
- Cập nhật `tail` thành nút mới.

🔍

Thêm vào vị trí bất kỳ:

- Duyệt đến vị trí mong muốn.
- Cập nhật con trỏ `next` và `prev` của nút mới và các nút lân cận.



🗑️

Xóa nút (Deletion)

➡

Xóa nút đầu:

- Cập nhật `head` trỏ tới `head->next`.
- Nếu `head` mới tồn tại, đặt `head->prev` thành `NULL`.
- Giải phóng bộ nhớ của nút đầu cũ.

➡

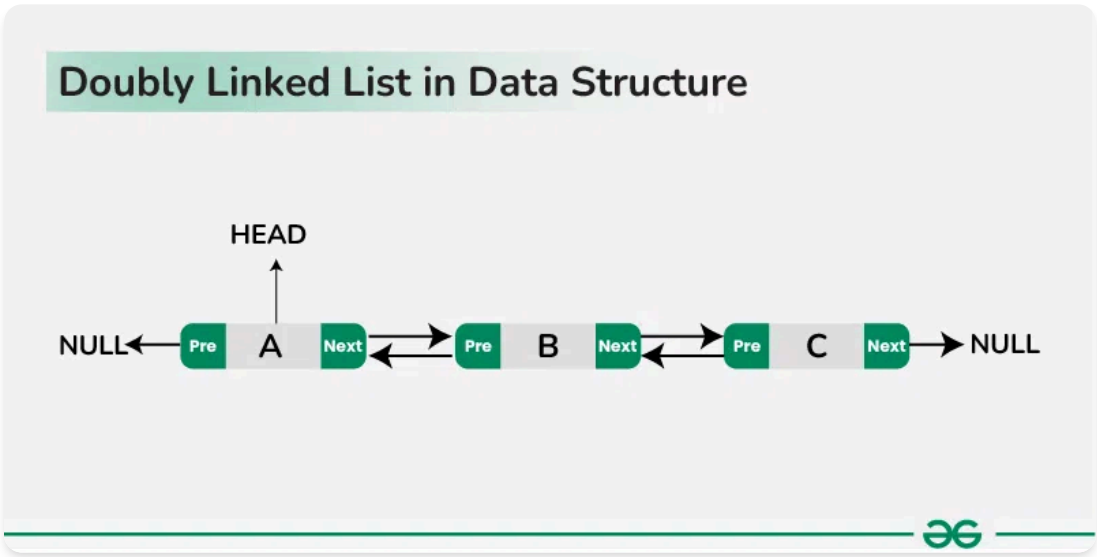
Xóa nút cuối:

- Cập nhật `tail` trỏ tới `tail->prev`.
- Nếu `tail` mới tồn tại, đặt `tail->next` thành `NULL`.
- Giải phóng bộ nhớ của nút cuối cũ.

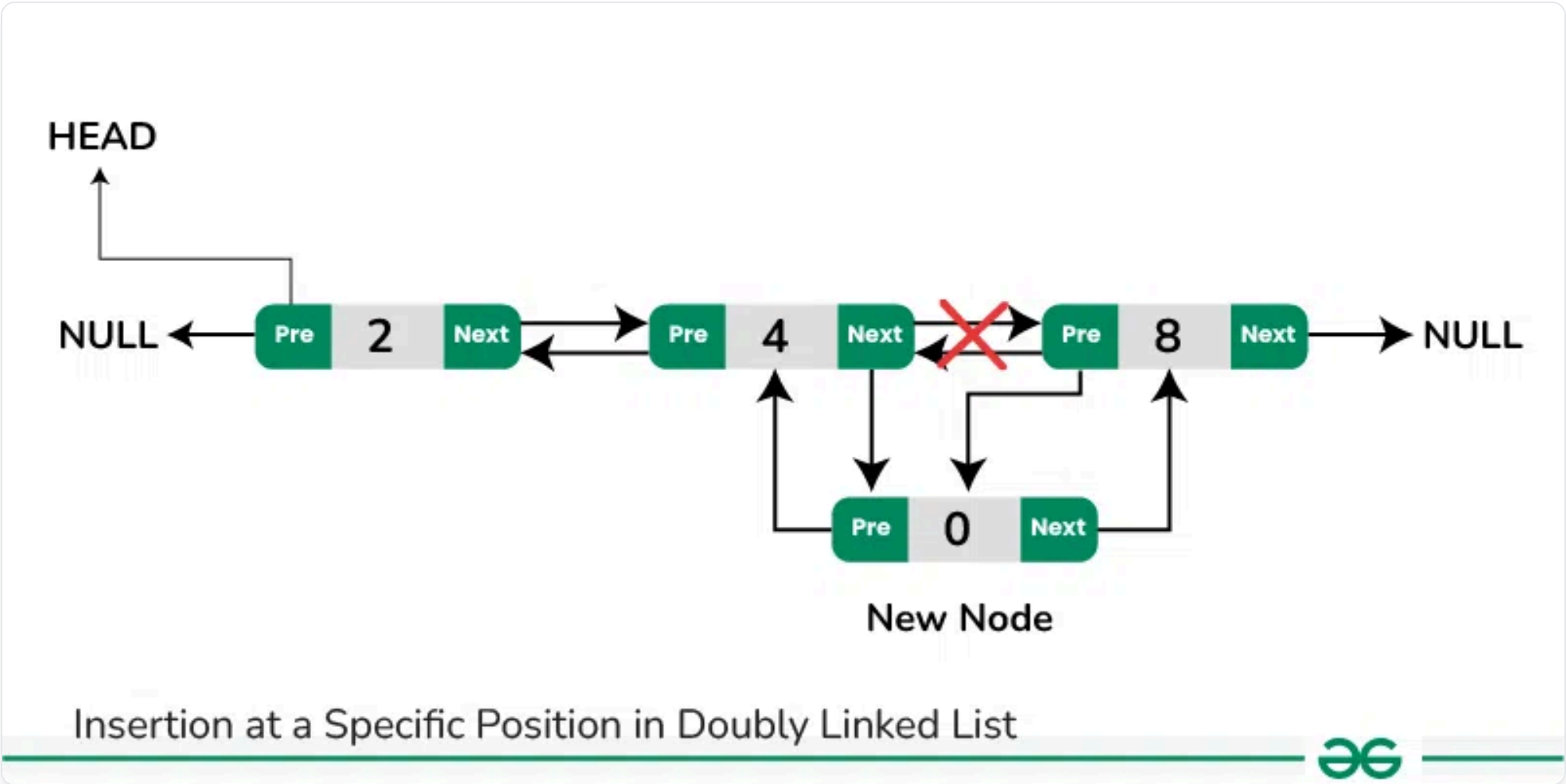
⊗

Xóa nút cụ thể:

- Tìm nút cần xóa.
- Cập nhật con trỏ `next` của nút trước và `prev` của nút sau.
- Giải phóng bộ nhớ của nút đã xóa.



4. Triển khai trong C++



Source: Doubly Linked List in C++ - GeeksforGeeks

🌟 Định nghĩa `Node`

Mục đích: Là thành phần cơ bản, đại diện cho một phần tử trong danh sách.

Các thành phần:

- `data`: Lưu trữ giá trị thực tế của nút.
- `next`: Con trỏ tới `Node` kế tiếp (nullptr nếu là nút cuối).
- `prev`: Con trỏ tới `Node` đứng trước (nullptr nếu là nút đầu).

Ví dụ cấu trúc:

```
struct Node {
    int data;
    Node* next;
    Node* prev;

    Node(int val) {
        data = val;
        next = nullptr;
        prev = nullptr;
    }
};
```

📋 Định nghĩa lớp `DoublyLinkedList`

Mục đích: Quản lý toàn bộ danh sách và cung cấp các phương thức thao tác.

Các thuộc tính chính:

- `head`: Con trỏ tới nút đầu tiên.
- `tail`: Con trỏ tới nút cuối cùng.

Các phương thức cơ bản (minh họa):

- `insertAtHead(val)`
- `insertAtTail(val)`
- `deleteNode(val)`
- `displayForward()`

Ví dụ cấu trúc lớp:

```
class DoublyLinkedList {
public:
    Node* head;
    Node* tail;

    DoublyLinkedList();
    void insertAtHead(int val);
    void insertAtTail(int val);
    // ... các phương thức khác
};
```

📌 Lưu ý quan trọng khi triển khai

- **Quản lý bộ nhớ:** Cấp phát (`new`) và giải phóng (`delete`) bộ nhớ là tối quan trọng để tránh rò rỉ bộ nhớ (memory leaks).
- **Xử lý trường hợp biên:** Luôn kiểm tra các trường hợp đặc biệt: danh sách rỗng, danh sách chỉ có một nút, thao tác ở đầu/cuối danh sách.
- **Tính linh hoạt:** Sử dụng `template` để tạo lớp `DoublyLinkedList` có thể làm việc với nhiều kiểu dữ liệu khác nhau (int, string, object,...).

5. Ví dụ mã C++: Thêm nút

+ Thêm nút vào đầu (insertAtBeginning)

Mã giả (Pseudocode)

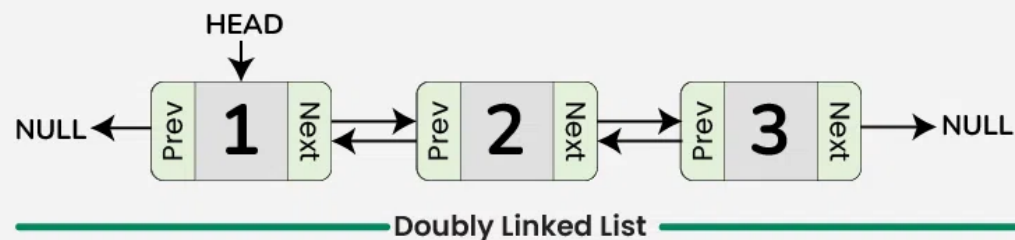
```
FUNCTION insertAtBeginning(newNode, head)
  IF head IS NULL THEN
    head = newNode
  ELSE
    newNode.next = head
    head.prev = newNode
    head = newNode
  END IF
END FUNCTION
```

Mã C++ chi tiết

```
void insertAtBeginning(Node*& head, int data) {
  Node* newNode = new Node(data); // 1. Tạo nút mới
  if (head == nullptr) { // 2. Nếu danh sách rỗng
    head = newNode;
  } else { // 3. Nếu danh sách không rỗng
    newNode->next = head;
    head->prev = newNode;
    head = newNode;
  }
}
```

Giải thích

- **Tạo nút mới:** Khởi tạo `newNode` với dữ liệu. `prev` và `next` là `nullptr`.
- **Kiểm tra rỗng:** Nếu danh sách rỗng (`head == nullptr`), `head` mới chính là `newNode`.
- **Cập nhật con trỏ:**
`newNode->next = head;` trỏ `next` của nút mới tới `head` cũ.
`head->prev = newNode;` trỏ `prev` của `head` cũ về nút mới.
`head = newNode;` cập nhật `head` của danh sách.



+ Thêm nút vào cuối (insertAtEnd)

Mã giả (Pseudocode)

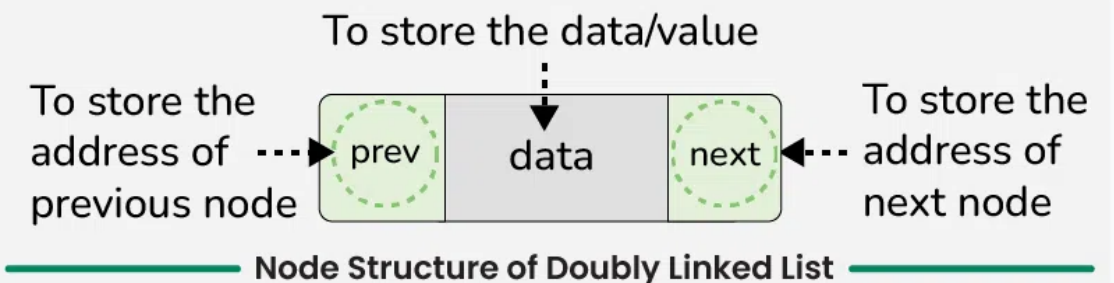
```
FUNCTION insertAtEnd(newNode, head, tail)
  IF head IS NULL THEN
    head = newNode
    tail = newNode
  ELSE
    tail.next = newNode
    newNode.prev = tail
    tail = newNode
  END IF
END FUNCTION
```

Mã C++ chi tiết

```
void insertAtEnd(Node*& head, Node*& tail, int data) {
  Node* newNode = new Node(data); // 1. Tạo nút mới
  if (head == nullptr) { // 2. Nếu danh sách rỗng
    head = newNode;
    tail = newNode;
  } else { // 3. Nếu danh sách không rỗng
    tail->next = newNode;
    newNode->prev = tail;
    tail = newNode;
  }
}
```

Giải thích

- **Tạo nút mới:** Khởi tạo `newNode` với dữ liệu.
- **Kiểm tra rỗng:** Nếu rỗng, cả `head` và `tail` đều trỏ đến `newNode`.
- **Cập nhật con trỏ:**
`tail->next = newNode;` trỏ `next` của `tail` cũ tới `newNode`.
`newNode->prev = tail;` trỏ `prev` của `newNode` về `tail` cũ.
`tail = newNode;` cập nhật `tail` của danh sách.



6. Ví dụ mã C++: Xóa nút

🗑️ Hàm `deleteNode(int key)`

Mã giả (Pseudocode)

```
FUNCTION deleteNode(head, tail, key)
  IF head IS NULL THEN
    RETURN // Danh sách rỗng
  END IF

  currentNode = head
  WHILE currentNode IS NOT NULL AND currentNode.data != key DO
    currentNode = currentNode.next
  END WHILE

  IF currentNode IS NULL THEN RETURN // Nút không tìm thấy

  // Cập nhật con trỏ prev của nút kế tiếp
  IF currentNode.next IS NOT NULL THEN
    currentNode.next.prev = currentNode.prev
  ELSE
    tail = currentNode.prev // Xóa nút cuối
  END IF

  // Cập nhật con trỏ next của nút trước đó
  IF currentNode.prev IS NOT NULL THEN
    currentNode.prev.next = currentNode.next
  ELSE
    head = currentNode.next // Xóa nút đầu
  END IF

  DELETE currentNode
END FUNCTION
```

Mã C++ chi tiết

```
void deleteNode(Node*& head, Node*& tail, int key) {
  if (head == nullptr) { // 1. Danh sách rỗng
    return;
  }

  Node* current = head;
  while (current != nullptr && current->data != key) {
    current = current->next;
  }

  if (current == nullptr) { // 2. Không tìm thấy nút
    return;
  }

  // 3. Xử lý cập nhật con trỏ
  if (current->next != nullptr) {
    current->next->prev = current->prev;
  } else { // current là nút cuối
    tail = current->prev;
  }

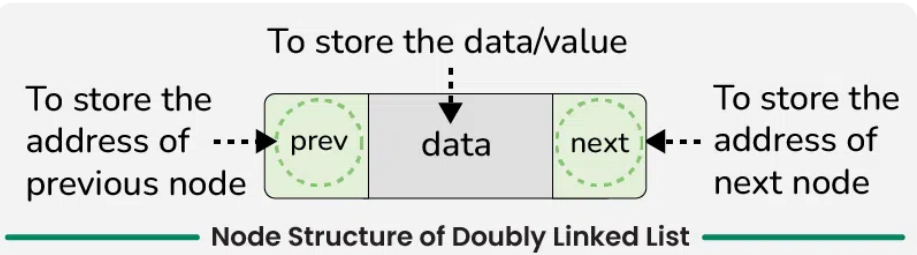
  if (current->prev != nullptr) {
    current->prev->next = current->next;
  } else { // current là nút đầu
    head = current->next;
  }

  if (head == nullptr) { // Danh sách trở thành rỗng
    tail = nullptr;
  }

  delete current;
}
```

⚙️ Xử lý các trường hợp đặc biệt

- 1. Danh sách rỗng:** Kiểm tra `head == nullptr` ngay từ đầu.
- 2. Nút không tìm thấy:** Sau khi duyệt, `current` vẫn là `nullptr`.
- 3. Xóa nút đầu:** Cập nhật `head` mới, `prev` của `head` mới là `nullptr`.
- 4. Xóa nút cuối:** Cập nhật `tail` mới, `next` của `tail` mới là `nullptr`.
- 5. Xóa nút giữa:** Nút trước trở tới nút sau, và ngược lại.
- 6. Danh sách trở nên rỗng:** Nếu xóa nút duy nhất, cả `head` và `tail` đều phải là `nullptr`.



Mình họa xóa nút ở giữa danh sách.
Nguồn: GeeksforGeeks, Programiz

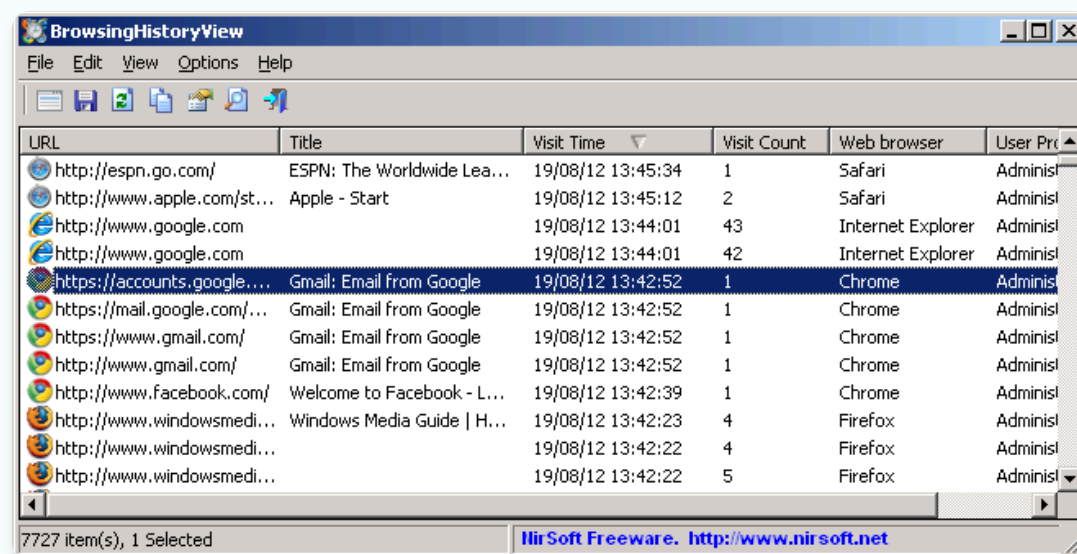
7. Các ứng dụng thực tế

↶ Tính năng Undo/Redo

- ✓ DLLs là xương sống cho các tính năng hoàn tác (Undo) và làm lại (Redo) trong nhiều ứng dụng.
- ✓ Mỗi hành động của người dùng được lưu trữ như một nút. Undo di chuyển con trỏ lùi (`prev`), Redo di chuyển tiến (`next`).
- ✓ Cho phép duyệt lịch sử thao tác hai chiều một cách linh hoạt và hiệu quả.

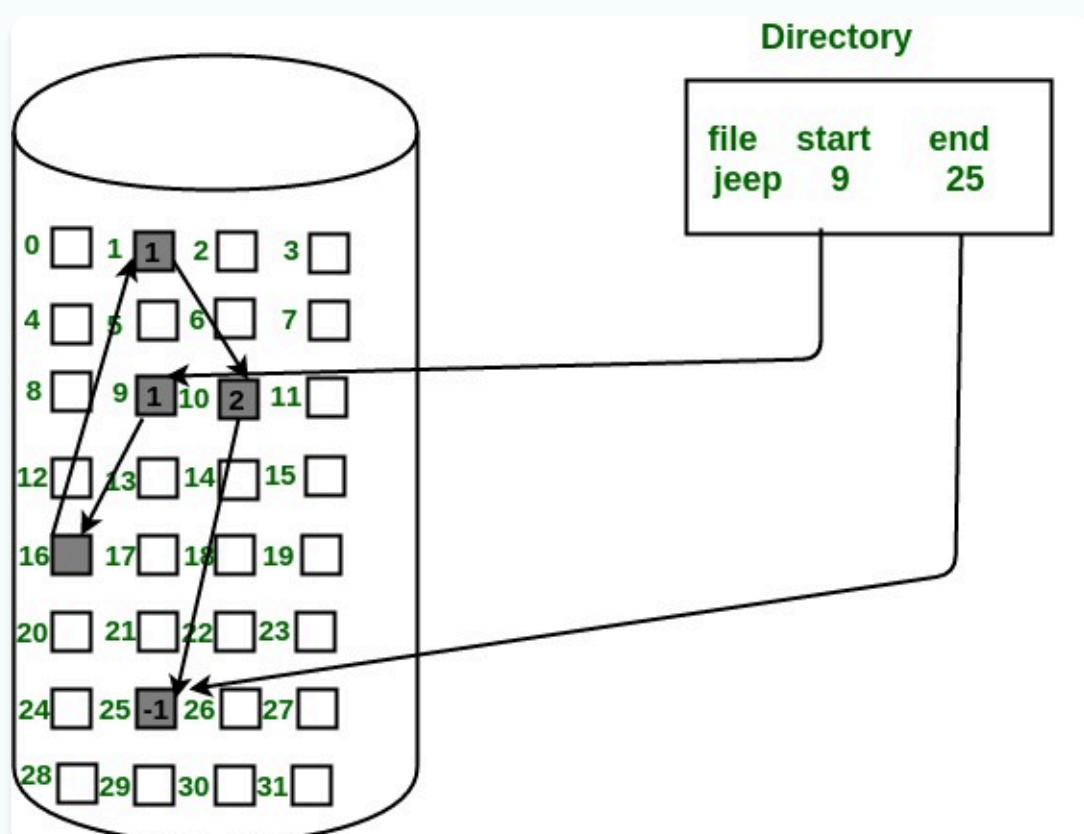
↶ Lịch sử duyệt web (Back/Forward)

Trình duyệt web dùng DLL để quản lý lịch sử trang. Mỗi trang là một nút, cho phép điều hướng tới/lui dễ dàng qua nút "Back" và "Forward".



🗄️ Quản lý bộ nhớ

Hệ điều hành dùng DLL để duy trì danh sách các khối bộ nhớ trống hoặc đã cấp phát, giúp phân bổ và thu hồi bộ nhớ hiệu quả.



🌳 Cấu trúc dữ liệu phức tạp

- ✓ DLL là nền tảng cho các cấu trúc phức tạp hơn như cây tìm kiếm nhị phân (BST), cây cân bằng (AVL, Red-Black Tree).
- ✓ Các con trỏ `parent` và `child` trong cây có thể hoạt động tương tự như `prev` và `next` để tối ưu hóa việc duyệt và thao tác trên cây.

⚙️ Cơ chế Undo/Redo chi tiết với Danh sách liên kết đôi

Lưu trữ hành động: Mỗi thao tác (gõ phím, cắt, dán) được đóng gói và thêm vào cuối một DLL, tạo thành một lịch sử thao tác.

Thực hiện Undo: Con trỏ lùi về nút trước đó (`prev`). Hệ thống thực hiện hành động ngược lại với lệnh tại nút đó.

Thực hiện Redo: Con trỏ tiến tới nút kế tiếp (`next`). Hệ thống thực hiện lại lệnh tại nút đó.

Thao tác mới sau Undo: Nếu có thao tác mới, tất cả các nút phía trước con trỏ (lịch sử "Redo") sẽ bị xóa để bắt đầu một nhánh lịch sử mới.

Tổng kết và hỏi đáp

Đặc điểm cốt lõi

Node Prev Next Duyệt 2 chiều

Cấu trúc dữ liệu tuyến tính với mỗi nút chứa dữ liệu, con trỏ `prev` và `next` trỏ đến nút trước và sau, cho phép di chuyển hai chiều.

Lợi thế chính

Linh hoạt Thêm/Xóa hiệu quả

Hỗ trợ duyệt tiến/lùi dễ dàng và các thao tác thêm/xóa nút ở bất kỳ vị trí nào một cách hiệu quả hơn so với danh sách liên kết đơn.

Hạn chế cần lưu ý

Tốn bộ nhớ Triển khai phức tạp

Yêu cầu thêm không gian bộ nhớ cho con trỏ `prev` và quá trình triển khai có thể phức tạp hơn do phải quản lý hai con trỏ.

Ứng dụng phổ biến

Undo/Redo Lịch sử duyệt web Quản lý bộ nhớ

Được ứng dụng rộng rãi trong các hệ thống cần lịch sử thao tác (Undo/Redo), duyệt trang (lịch sử web) và quản lý tài nguyên hệ thống.



Thảo luận & Hỏi đáp